

Neuroevolution: búsqueda de arquitectura de red neuronal con AG

Guía de explicación oral — Integrante 4 · Notebook 03_neuroevolution.ipynb

Este documento explica, paso a paso y en el mismo orden en que aparece en el notebook, cómo funciona el algoritmo genético (AG) que busca la mejor **arquitectura** de una red neuronal (MLPClassifier de scikit-learn) para el dataset **Breast Cancer Wisconsin (Diagnostic)**. La idea central que debo transmitir al docente es: *el AG no entrena la red, el AG decide qué red construir; una vez fijada la arquitectura, el MLP aprende sus pesos normalmente mediante backpropagation.*

Orden sugerido de la explicación oral (regla de oro del equipo)

- **1. Qué optimizo** → la arquitectura de una red neuronal (capas, neuronas, activación, regularización).
- **2. Cómo lo represento** → un cromosoma de 5 genes: [n_capas, n1, n2, activación, alpha].
- **3. Cómo calculo fitness** → accuracy promedio con validación cruzada (3 folds) sobre el set de entrenamiento.
- **4. Cómo evoluciona** → selección por torneo, cruzamiento por gen, mutación con probabilidad baja, elitismo.
- **5. Qué resultado obtuve** → mejor arquitectura encontrada y su accuracy final sobre test.

1. Contexto y dataset compartido por el equipo

Los tres notebooks del equipo (Feature Selection, Hyperparameter Optimization y Neuroevolution) usan **el mismo dataset y el mismo split**, para que los resultados sean comparables entre sí. En vez de cargar `load_breast_cancer()` por separado en cada notebook, se usa una función compartida `cargar_datos_base()` definida por el Integrante 1 en `ga_utils.py`, dentro del repositorio de GitHub del equipo.

```
import warnings
warnings.filterwarnings("ignore")

import random
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import cross_val_score
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

RANDOM_STATE = 42
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)
```

Se fija una semilla (`RANDOM_STATE = 42`) para que la demostración sea **reproducible**: si el docente pide repetir la ejecución, los resultados no deberían cambiar drásticamente.

```
!git clone https://github.com/JoseChavez15/Algoritmos-Geneticos-ML.git
%cd Algoritmos-Geneticos-ML

import sys
sys.path.append('.')
from ga_utils import cargar_datos_base

X_train, X_test, y_train, y_test, feature_names, target_names = \
    cargar_datos_base(seed=RANDOM_STATE, test_size=0.20)
```

Regla metodológica clave (para defender en la evaluación oral): el conjunto de **test (20%)** nunca se usa durante la evolución. El AG solo ve el conjunto de **train (80%)**. Esto evita "data leakage" y hace que la accuracy final en test sea una medida honesta del desempeño.

2. Representación del cromosoma

A diferencia de Feature Selection (máscara binaria) o Hyperparameter Optimization (parámetros de un Random Forest), aquí cada individuo es un conjunto de **decisiones arquitectónicas** para una red neuronal:

Gen	Significado	Valores posibles
n_capas	Número de capas ocultas	1 ó 2
n1	Neuronas en la capa 1	8, 16, 32, 64
n2	Neuronas en la capa 2 (0 si n_capas=1)	0, 8, 16, 32
activación	Función de activación de las neuronas	relu, tanh, logistic

alpha	Regularización L2 del MLP	1e-4, 1e-3, 1e-2
-------	---------------------------	------------------

Ejemplo de cromosoma: [2, 32, 16, relu, 0.001] → red con dos capas ocultas (32 y 16 neuronas), activación ReLU, alpha = 0.001.

```
SEARCH_SPACE = {
    'n_capas': [1, 2],
    'n1': [8, 16, 32, 64],
    'n2': [0, 8, 16, 32],
    'activacion': ['relu', 'tanh', 'logistic'],
    'alpha': [1e-4, 1e-3, 1e-2],
}

def crear_individuo():
    n_capas = random.choice(SEARCH_SPACE['n_capas'])
    n1 = random.choice(SEARCH_SPACE['n1'])
    n2 = random.choice(SEARCH_SPACE['n2']) if n_capas == 2 else 0
    activacion = random.choice(SEARCH_SPACE['activacion'])
    alpha = random.choice(SEARCH_SPACE['alpha'])
    return {'n_capas': n_capas, 'n1': n1, 'n2': n2,
            'activacion': activacion, 'alpha': alpha}

def cromosoma_a_hidden_layers(ind):
    if ind['n_capas'] == 1 or ind['n2'] == 0:
        return (ind['n1'],)
    return (ind['n1'], ind['n2'])
```

crear_individuo() genera un cromosoma aleatorio respetando el espacio de búsqueda (inicialización de la población). **cromosoma_a_hidden_layers()** traduce el cromosoma al formato que espera scikit-learn (`hidden_layer_sizes`), por ejemplo (32, 16).

3. Función de fitness

Cada individuo (arquitectura) se convierte en un Pipeline con StandardScaler + MLPClassifier, y se evalúa con **validación cruzada de 3 folds sobre el train**. Se usa `early_stopping=True` y un `max_iter` moderado para que cada evaluación sea rápida, ya que durante la evolución se entrenan decenas de redes.

```
def calcular_fitness(ind, X_tr, y_tr, cv=3):
    hidden = cromosoma_a_hidden_layers(ind)
    modelo = Pipeline([
        ('scaler', StandardScaler()),
        ('mlp', MLPClassifier(
            hidden_layer_sizes=hidden,
            activation=ind['activacion'],
            alpha=ind['alpha'],
            early_stopping=True,
            max_iter=300,
            random_state=RANDOM_STATE,
        )),
    ])
    try:
        scores = cross_val_score(modelo, X_tr, y_tr, cv=cv, scoring='accuracy')
        return scores.mean()
    except Exception:
        return 0.0 # penaliza combinaciones que fallan al entrenar
```

El StandardScaler es importante para explicar oralmente: las redes neuronales convergen mejor cuando las variables tienen media 0 y varianza 1; sin escalar, columnas con rangos grandes dominarían el entrenamiento.

4. Operadores genéticos

Selección — torneo de 3: se eligen 3 individuos al azar y gana el de mejor fitness. **Cruzamiento — por gen:** cada gen del hijo se toma al azar de uno de los dos padres. **Mutación:** cada gen tiene una probabilidad baja (20%) de ser reemplazado por otro valor válido del espacio de búsqueda. En ambos casos, si `n_capas == 1`, se corrige `n2 = 0` para que el cromosoma siga siendo válido.

```
def seleccion_torneo(poblacion, fitnesses, k=3):
    participantes = random.sample(range(len(poblacion)), k)
    mejor_idx = max(participantes, key=lambda i: fitnesses[i])
    return poblacion[mejor_idx]

def cruzamiento(padre1, padre2):
    hijo = {gene: random.choice([padre1[gene], padre2[gene]]) for gene in padre1}
    if hijo['n_capas'] == 1:
        hijo['n2'] = 0
    return hijo

def mutacion(ind, p_mut=0.2):
    nuevo = dict(ind)
    for gene in SEARCH_SPACE:
        if gene == 'n2' and nuevo['n_capas'] == 1:
            continue
        if random.random() < p_mut:
            nuevo[gene] = random.choice(SEARCH_SPACE[gene])
    if nuevo['n_capas'] == 1:
        nuevo['n2'] = 0
```

return nuevo

5. Ciclo genético completo

Población de 20 individuos, 10 generaciones, mutación con probabilidad 0.2. Se aplica **elitismo simple**: el mejor individuo de cada generación pasa directo a la siguiente, para no perder la mejor solución encontrada.

```
POP_SIZE = 20
N_GEN = 10
P_MUT = 0.2

poblacion = [crear_individuo() for _ in range(POP_SIZE)]
historial_mejor_fitness = []
mejor_individuo_global = None
mejor_fitness_global = -1

for gen in range(N_GEN):
    fitnesses = [calcular_fitness(ind, X_train, y_train) for ind in poblacion]

    idx_mejor = int(np.argmax(fitnesses))
    mejor_fitness_gen = fitnesses[idx_mejor]
    mejor_individuo_gen = poblacion[idx_mejor]
    historial_mejor_fitness.append(mejor_fitness_gen)

    if mejor_fitness_gen > mejor_fitness_global:
        mejor_fitness_global = mejor_fitness_gen
        mejor_individuo_global = mejor_individuo_gen

    nueva_poblacion = [mejor_individuo_gen] # elitismo
    while len(nueva_poblacion) < POP_SIZE:
        padre1 = seleccion_torneo(poblacion, fitnesses)
        padre2 = seleccion_torneo(poblacion, fitnesses)
        hijo = cruzamiento(padre1, padre2)
        hijo = mutacion(hijo, P_MUT)
        nueva_poblacion.append(hijo)

    poblacion = nueva_poblacion
```

Este bloque contiene **todo el ciclo genético** que la rúbrica pide señalar: inicialización de la población, evaluación de fitness, selección, cruzamiento, mutación, nueva población y criterio de terminación (número fijo de generaciones).

6. Gráfica de evolución

```
plt.figure(figsize=(8, 5))
plt.plot(range(1, N_GEN + 1), historial_mejor_fitness, marker='o')
plt.xlabel("Generación")
plt.ylabel("Mejor fitness (accuracy CV)")
plt.title("Evolución del fitness - Neuroevolution")
plt.grid(True)
plt.show()
```

Esta gráfica es la evidencia visual de que el AG está **aprendiendo**: se espera que el mejor fitness se mantenga igual o mejore generación a generación (nunca empeora gracias al elitismo).

7. Evaluación final en test

Una vez terminada la evolución, se toma la **mejor arquitectura encontrada** y se entrena un modelo final con todo el train, evaluándolo sobre el 20% de test que nunca participó en la búsqueda.

```
hidden_final = cromosoma_a_hidden_layers(mejor_individuo_global)

modelo_final = Pipeline([
    ('scaler', StandardScaler()),
    ('mlp', MLPClassifier(
        hidden_layer_sizes=hidden_final,
        activation=mejor_individuo_global['activacion'],
        alpha=mejor_individuo_global['alpha'],
        early_stopping=True,
        max_iter=300,
        random_state=RANDOM_STATE,
    )),
])

modelo_final.fit(X_train, y_train)
y_pred = modelo_final.predict(X_test)
acc_test = accuracy_score(y_test, y_pred)
```

Comparación contra un MLP por defecto

Para demostrar que la búsqueda evolutiva realmente aporta algo, se entrena también un MLP sin optimizar (arquitectura por defecto de scikit-learn) y se comparan ambos resultados sobre el mismo test.

```
modelo_base = Pipeline([
    ('scaler', StandardScaler()),
    ('mlp', MLPClassifier(early_stopping=True, max_iter=300, random_state=RANDOM_STATE)),
])
modelo_base.fit(X_train, y_train)
y_pred_base = modelo_base.predict(X_test)
acc_base = accuracy_score(y_test, y_pred_base)
```

8. Resultado obtenido y conclusión

Elemento	Valor
Mejor arquitectura (hidden_layer_sizes)	(64, 8)
Activación	relu
Alpha (regularización)	0.01
Mejor fitness en validación (train, CV)	0.9604
Accuracy final en test	0.9604

Conclusión para exponer (2-4 líneas): el algoritmo genético evolucionó decisiones estructurales de la red (número de capas, neuronas por capa, activación y regularización), sin tocar los pesos internos del modelo. La mejor arquitectura encontrada igualó o mejoró el desempeño de un MLP por defecto usando un espacio de búsqueda pequeño y pocas generaciones, lo que confirma que incluso una búsqueda evolutiva simple puede guiar de forma efectiva el diseño de una red neuronal.

9. Preguntas que debo poder responder en la exposición

P: ¿Qué representa `hidden_layer_sizes=(32,16)`?

R: Una red con dos capas ocultas: 32 neuronas en la primera capa, 16 en la segunda.

P: ¿Por qué se escala la información antes de entrenar una MLP?

R: Las redes neuronales convergen mejor y más rápido cuando las variables tienen media 0 y varianza 1; sin escalar, las columnas con rangos grandes dominarían el entrenamiento.

P: ¿Cuál es la diferencia entre evolucionar arquitectura y entrenar pesos?

R: El AG elige la "forma" de la red (capas, neuronas, activación, alpha) sin tocar los pesos; una vez fijada la arquitectura, los pesos se aprenden con el optimizador normal del MLP (backpropagation).

P: ¿Por qué conviene limitar el espacio de búsqueda para esta demostración?

R: Cada evaluación de fitness implica entrenar una red completa con validación cruzada; un espacio muy grande o generaciones muy largas harían la demostración demasiado lenta para mostrarla en clase.

Cómo presentarlo oralmente (guía del equipo)

Duración sugerida: 3 a 4 minutos. Primero mostrar/dibujar la arquitectura que representa un cromosoma (ej. capas de 32 y 16 neuronas). Aclarar que el MLP sigue aprendiendo sus pesos con su optimizador normal; el AG solo busca qué arquitectura entrenar. Luego mostrar la gráfica de evolución del fitness y cerrar presentando la arquitectura ganadora y su accuracy en test.

Frase sugerida: "La neuroevolución de este ejemplo no reemplaza el entrenamiento del MLP: evoluciona decisiones arquitectónicas. Cada individuo se convierte en una red, se entrena y su desempeño actúa como fitness."